

What is React?

Real-World Analogy

Imagine building a large apartment complex.

Without React:

- Every time a tenant changes a light bulb, you rebuild the entire building.
- Every time someone moves out, you repaint the whole structure.

With React:

- Each apartment is independent.
- If something changes inside one apartment, only that apartment updates.

React is like:

- A smart building management system
- That tracks exactly what changed
- And updates only that part

WHY React exists:

Traditional DOM manipulation was slow and messy.

React solves:

- Manual DOM updates
- State complexity
- UI synchronization issues

WHAT React is:

A **JavaScript library for building user interfaces**, specifically:

- Component-based
- Declarative
- Efficient via Virtual DOM

HOW React works:

1. You describe UI as a function of state.
2. React builds a Virtual DOM.
3. It compares old vs new (diffing).
4. Updates only what's necessary (reconciliation).

Creating React App (Modern Way)

`npm create vite@latest my-app`

`cd my-app`

`npm install`

`npm run dev`

Walking through react app structure

my-app/

```
|  
|  
| └─ node_modules/  
| └─ public/  
| └─ src/  
|   | └─ assets/  
|   | └─ components/  
|   | └─ App.jsx  
|   | └─ main.jsx  
|   └─ index.css
```

```
|  
├── .gitignore  
├── index.html  
├── package.json  
└── vite.config.js
```

1 package.json — The Brain of the Project

What it does:

- Lists dependencies (React, libraries, etc.)
- Defines scripts (npm run dev, build)
- Contains project metadata

2 node_modules/ — Installed Libraries

Contains all installed packages.

3 public/ — Static Files

Used for:

- Images
- Favicon
- Static files

Files here are not processed by React.

4 index.html — The Single HTML Page

This is important.

React apps are Single Page Applications (SPA).

```
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
```

👉 The #root div is where React injects everything.

5 src/ — Where Real Work Happens

This is your actual app code.

6 main.jsx — Entry Point

This is where React starts.

```
import React from "react"
import ReactDOM from "react-dom/client"
import App from "./App"
```

```
ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
)
```

What it does:

1. Finds #root in index.html
2. Renders <App /> inside it

index.html → main.jsx → App.jsx

7 App.jsx — Root Component

This is your main component.

```
function App() {  
  return (  
    <div>  
      <h1>Hello React</h1>  
    </div>  
  )  
}
```

export default App

Everything inside your app eventually connects to this.

Think of it as:

The top-level parent of your entire component tree.

components/ — Reusable UI Parts

Example:

components/
 Navbar.jsx
 Button.jsx
 Card.jsx

assets/ — Images, Fonts, etc.

For importing inside React:

```
import logo from "./assets/logo.png"
```

```
<img src={logo} />
```

How Everything Connects (Flow Diagram)

package.json → installs React

index.html → has #root

main.jsx → mounts React

App.jsx → root component

components/ → child components

Q1: Why React over vanilla JS?

Because React abstracts DOM manipulation, improves maintainability, and introduces component-based architecture.

Q.2 What is virtual DOM

An in-memory JavaScript representation of the real DOM used by React to efficiently calculate minimal UI updates through a diffing algorithm before applying changes to the actual DOM.

Instead of updating the real browser DOM directly, React:

1. Creates a JS copy of the UI (Virtual DOM)
2. Compares it with the previous version
3. Updates only what changed in the real DOM

Real DOM = Expensive to update

Virtual DOM = Cheap JS object

Q.3 Real DOM vs Virtual DOM

Feature	Real DOM	Virtual DOM
Lives where?	Browser	JavaScript memory
Update speed	Slow (reflow/repaint)	Fast comparison
Control style	Imperative	Declarative
Developer effort	Manual DOM handling	React manages updates
Performance strategy	Direct mutation	Diff & patch

Q.4 How react works and why react is faster and efficient?

Step 1 — React creates a NEW Virtual DOM tree

Step 2 — React compares:

Old Virtual DOM

vs

New Virtual DOM

This comparison process is called:

👉 Reconciliation

Step 3 — React finds the difference

If only text changed:

`<h1>1</h1>`

`<h1>2</h1>`

React updates only the text node.

Not the whole page.

🧠 Why Is This Faster?

Because:

- JS object comparison is fast
- DOM manipulation is slow
- Layout + repaint is expensive

React minimizes real DOM touches.

React is not fast because of Virtual DOM alone.

It's fast because of:

- Smart diffing algorithm ($O(n)$)
- Batching updates
- Fiber architecture

Q.5: Is React a framework?

No, it's a library (focuses only on UI layer).

JSX

JSX (JavaScript XML.) is like writing HTML inside JavaScript — but smarter.

Instead of:

`document.createElement()`

You write:

`<div>Hello</div>`

It's syntactic sugar for `React.createElement()`.

How to write js and html

{here js} in bracket and simple html

Q. Can browsers understand JSX? (No)?

browsers cannot directly understand JSX. JSX is not valid JavaScript

So in short: **JSX → transpiled → JavaScript → browser understands it.**

`const element = <h1>Hello, world!</h1>;` to

`const element = React.createElement("h1", null, "Hello, world!");`

Q. Why use JSX?

Improved readability

Structured UI

JS + HTML together

Functional Components

A **functional component** is a **JavaScript function** that returns JSX (or React elements) to define part of the UI.

```
function Greeting() {  
  return <h1>Hello, World!</h1>;  
}
```

Or using arrow function syntax:

```
const Greeting = () => <h1>Hello, World!</h1>;
```

Q.Why functional Components over class based?

1. Simpler Syntax & Readability
2. Hooks Replace Lifecycle Methods

Class components rely on multiple lifecycle methods (componentDidMount, componentDidUpdate, componentWillUnmount) for state and side effects. This can get verbose and fragmented.

- 3.No this Binding Needed

In class components, you often need to bind this for event handlers:

4. Easier to Test & Reuse

- Functional components are plain functions, making them easier to unit-test.
- Hooks allow you to create custom reusable logic (custom hooks) without copying class lifecycles.

5. Performance Benefits

- Functional components are lighter; they don't need the overhead of class instantiation.
- React optimizations (like React.memo) are easier to use with functional components for preventing unnecessary re-renders.

-

Functional components are simpler, cleaner, more powerful, and future-proof, while class components are mostly maintained for older codebases.

PROPS vs STATE

Concept Definition

Props	Short for properties. Read-only data passed from parent to child component. Used to make components dynamic and reusable.
State	Local data managed inside a component. Can change over time, usually in response to user interaction, API calls, or other events.

Props Examples

```
function Greeting({ name, age }) {  
  return <h1>Hello, {name}! You are {age} years old.</h1>;  
}
```

// Usage in parent

```
function App() {  
  return <Greeting name="Prajwal" age={24} />;  
}
```

✅ Props are **read-only**:

```
function Greeting({ name }) {  
  // ❌ This is NOT allowed  
  // name = "Someone else"; // Error  
  return <h1>Hello, {name}</h1>;  
}
```

Props are **immutable in the child**; they can only change if the parent sends new values.

4. State Examples

```
import { useState } from "react";
```

```
function Counter() {
```

```

const [count, setCount] = useState(0);

const increment = () => setCount(count + 1);

return (
  <div>
    <p>Count: {count}</p>
    <button onClick={increment}>Increment</button>
  </div>
);
}

```

✓ State is **mutable within the component**:

- Updated using setCount (functional components) or this.setState (class components).
- Changing state triggers a **re-render**.

Feature	Props	State
Mutability	Immutable (read-only inside the child)	Mutable (can be changed using setState or useState)
Who controls it	Controlled by parent component	Controlled by the component itself
Purpose	Pass data and callbacks to children	Manage internal data and dynamic behavior
Lifecycle	Exists as long as parent passes it	Exists as long as component is mounted
Re-rendering	Component re-renders when props change	Component re-renders when state changes
Usage	Customization and communication	Dynamic UI, user interaction, timers, forms

5. Props vs State Interaction

Props can be used to **initialize state**, but state is independent afterward:

```
function Timer({ start }) {  
  const [count, setCount] = useState(start); // initialize with prop  
  
  const increment = () => setCount(count + 1);  
  
  return (  
    <div>  
      <p>Timer: {count}</p>  
      <button onClick={increment}>+1</button>  
    </div>  
  );  
}  
  
// Usage  
<Timer start={10} />
```

- start is a prop → **cannot be changed by Timer**
- count is state → **can be updated locally**

Common Rules

1. **Props are for passing data; state is for managing internal changes.**
2. **Do not modify props** inside a child component.
3. State updates are **asynchronous**, so you may need functional updates:

```
setCount(prev => prev + 1);
```

✓ Summary

- **Props:** Read-only, parent → child, static during render.
- **State:** Mutable, local to component, changes trigger re-render.
- Props can **initialize state**, but cannot be changed by the child.
- Together, they **enable dynamic, reusable, and interactive UIs** in React.

One Way Flow in react

Data in React **always flows from parent to child** via props.

A child component **cannot directly modify the parent's data**; it can only request changes via callbacks.

Q. Why Props Are Read-Only

Props are read-only because React enforces a **one-way data flow**:

- **Parent owns the data** (state or static values).
- **Child receives data via props.**
- Child cannot directly change the parent's data; it can only **read and render** it.

Reason for read-only:

1. **Predictability:** Data always comes from the parent → easier to debug.
2. **Single source of truth:** The parent is the only one responsible for data changes.
3. **Avoid side effects:** If children could mutate props, multiple components could cause unexpected changes.

Q. What Happens When State Updates?

React schedules a **state update**.

React compares the **previous virtual DOM** and the **new virtual DOM** (diffing).

React updates only the **actual DOM nodes that changed** (efficient rendering).

Component re-renders with **new state**, and the updated UI is displayed.

State is **local to the component**.

Calling `setState` or `useState` **doesn't immediately mutate** state; it queues an update.

Multiple state updates may be **batched** for performance.

Q. How to Update Parent From Child

Since **props are read-only**, children cannot directly change parent state. Instead, the **parent passes a function (callback) as a prop**, and the child calls it.

Q. What is Component Reusability?

Component reusability means creating React components that can be **used in multiple places** with **different data or behavior**, without rewriting code.

How Reusability Works in React

The core idea is **separating structure, behavior, and data**:

1. **Structure** → JSX layout.
2. **Behavior** → Functions, event handlers, or hooks.
3. **Data** → Passed via **props**, or handled via state/hooks.

Controlled vs Uncontrolled Components

Controlled Components

- The input's value is tied to React state.
- Every change is handled via an onChange event.
- React controls what is displayed.

```
import { useState } from "react";
```

```
function ControlledInput() {  
  const [text, setText] = useState("");
```

```
  const handleChange = (e) => setText(e.target.value);
```



```
const handleSubmit = () => alert(`Submitted: ${text}`);

return (
  <div>
    <input type="text" value={text} onChange={handleChange} />
    <button onClick={handleSubmit}>Submit</button>
    <p>Typed: {text}</p>
  </div>
);
}
```

Key Points:

- `value={text}` → input value is controlled by React state.
- Any change triggers `handleChange`, which updates state → UI updates.
- React has full control of the input.

Uncontrolled Components

- The input's value is stored in the DOM, not React.
- React reads the value only when needed, usually via a ref.

Example:

```
import { useRef } from "react";

function UncontrolledInput() {
  const inputRef = useRef();

  const handleSubmit = () => {
    alert(`Submitted: ${inputRef.current.value}`);
  };

  return (
    <div>
      <input type="text" ref={inputRef} />
      <button onClick={handleSubmit}>Submit</button>
    </div>
  );
}
```

```
</div>  
);  
}
```

Key Points:

- React does not track every keystroke.
- Value is accessed via ref on demand.
- Less code for simple forms but less control over behavior.

Feature	Controlled	Uncontrolled
State management	React state	DOM state
Updates	onChange handler	No automatic updates
Value access	Direct from state	Via ref (current.value)
Validation	Easy in React	Must access value manually
Predictability	High (React is source of truth)	Lower
Best for	Complex forms, validation, dynamic behavior	Simple forms, quick prototypes

5. When to Use Which

Controlled Components:

- **When you need to validate input dynamically.**
- **When UI depends on input (like enabling/disabling a button).**
- **When input is shared or synchronized across components.**

Uncontrolled Components:

- **Quick forms where React doesn't need to manage every keystroke.**
- **Migrating legacy code that uses plain HTML forms.**
- **Simple inputs with occasional value reading.**

Two Way Binding

Two-way data binding is a mechanism where:

Changes in the **UI** automatically update the **data model**, and changes in the **data model** automatically update the **UI**.

How React Handles Data Binding

React **does not have automatic two-way binding**. Instead, it uses **one-way data flow** with **controlled components** to mimic it.

```
import { useState } from "react";

function ControlledInput() {
  const [name, setName] = useState("");
  return (
    <div>
      <input value={name} onChange={(e) => setName(e.target.value)} />
      <p>Hello, {name}</p>
    </div>
  );
}
```

- Input value is **controlled by React state** (value={name})
- Every keystroke triggers onChange → setName updates state → component re-renders → UI updates.

✅ Effectively, it behaves like two-way binding, but **React explicitly manages the update**. So React is **explicitly one-way**, but you can achieve the effect of two-way binding manually.

List and Key

lists are usually created by mapping over arrays to generate multiple components dynamically.

```
const fruits = ["Apple", "Banana", "Mango"];
```

```
function FruitList() {  
  return (  
    <ul>  
      {fruits.map((fruit, index) => (  
        <li key={fruit}>{fruit}</li>  
      ))}  
    </ul>  
  );  
}
```

Here, `.map()` generates an `<li>` for each fruit.

This is a dynamic list rendered based on data.

Why Keys Are Important

React needs a way to identify which items have changed, added, or removed when re-rendering lists.

- Key = unique identifier for each list item
- Helps React optimize re-rendering efficiently.
- Without keys, React may re-render unnecessarily or incorrectly.

What Happens Without Keys

- React warns: Each child in a list should have a unique "key" prop.
- Without keys, React will:
 - Re-render all list items unnecessarily
 - Potentially reuse wrong DOM elements → UI bugs

Rules for Keys

1. Unique among siblings
 - Can't use the same key for multiple items in the same list.
2. Stable
 - Keys should not change between renders if the item stays in the list.
3. Avoid using index as key if items can change order

Using the index as a key can confuse React when items are added, removed, or moved, so it might show the wrong data. It's safer to use a unique ID for each item.
4. Prefer unique IDs from data if available:

Conditional Rendering

Conditional Rendering means rendering different UI elements based on a condition.

- React lets you use JavaScript conditions inside JSX to decide what to display.

How to Do Conditional Rendering

Using if statements

```
function Greeting({ isLoggedIn }) {  
  if (isLoggedIn) {  
    return <h1>Welcome back!</h1>;  
  } else {  
    return <h1>Please sign in.</h1>;  
  }  
}
```

b) Using Ternary Operator

```
function Greeting({ isLoggedIn }) {  
  return <h1>{isLoggedIn ? "Welcome back!" : "Please sign in."}</h1>;  
}
```

c) Using Logical AND &&

```
function Notification({ message }) {  
  return <div>{message && <p>{message}</p>}</div>;  
}
```

- If message exists → display <p>
- If message is falsy → render nothing

d) Using Switch Statements

```
function Status({ status }) {  
  switch(status) {  
    case "loading":  
      return <p>Loading...</p>;  
    case "success":  
      return <p>Data loaded!</p>;  
    case "error":  
      return <p>Error occurred!</p>;  
    default:  
      return null;  
  }  
}
```

3. Why Conditional Rendering?

- Show **different content** based on app state
- Hide or display elements dynamically
- Keep the **UI clean and responsive**

Event Handling

- React events are similar to HTML events (like onclick), but written in **camelCase** and passed as **functions**.
- You **don't use strings**, like in HTML, e.g., onclick="doSomething()".

```
function Button() {  
  const handleClick = () => {  
    alert("Button clicked!");  
  };  
  
  return <button onClick={handleClick}>Click Me</button>;  
}
```

3. Passing Arguments to Event Handlers

```
function Button({ label }) {  
  const handleClick = (msg) => alert(msg);  
  
  return <button onClick={() => handleClick(`You clicked  
    ${label}`)}>{label}</button>;  
}
```

- Use **arrow function** to pass arguments
- Avoid calling the function directly like onClick={handleClick("Hi")} (runs immediately)

Q. “Why use `setCount(prev => prev + 1)` instead of `setCount(count + 1)`?”

When updating state based on the previous value, it's safer to use the **functional form** of `setState` (e.g., `setCount(prev => prev + 1)`) because React may batch multiple state updates. Using the variable directly (`setCount(count + 1)`) can lead to **stale state bugs** when multiple updates happen in the same render.

```
import { useState } from "react";
```

```
function Counter() {
```

```
  const [count, setCount] = useState(0);
```

```
  // Potential bug: count may not increase correctly if clicked multiple times quickly
```

```
  const handleDirect = () => {
```

```
    setCount(count + 1);
```

```
    setCount(count + 1); // Only increments by 1, not 2
```

```
  };
```

```
  // Correct approach using functional updater
```

```
  const handleFunctional = () => {
```

```
    setCount(prev => prev + 1);
```

```
    setCount(prev => prev + 1); // Correctly increments by 2
```

```
  };
```

```
  return (
```

```
    <div>
```

```
      <h1>Count: {count}</h1>
```

```
    <button onClick={handleDirect}>Increment Directly</button>
    <button onClick={handleFunctional}>Increment Safely</button>
  </div>
);
}

export default Counter;
```